

第 31 讲

PyTorch Training Loop: `zero_grad` → Forward → Loss → Backward → Step

把 Module、Autograd、Loss 与 Optimizer 接成真正的训练闭环

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 26 讲已经用 NumPy 手写过完整 Training Loop；第 27–30 讲又分别学了 Tensor、Linear/Activation、Autograd 与 nn.Module。现在把这些零件重新装回 PyTorch 的标准训练循环，并逐步观察：哪一步算预测、哪一步算 Loss、哪一步只算梯度、哪一步才真正修改 Parameter。

学习目标

学完本讲，你应该能够：

1. 写出最小、正确的 PyTorch Training Loop；
2. 解释 `optimizer.zero_grad()` 为什么需要；
3. 区分 Forward、Loss、Backward、Optimizer Step 各自的职责；
4. 解释 `loss.backward()` 为什么不会修改 Weight；
5. 解释 `optimizer.step()` 到底依据什么修改参数；
6. 理解 PyTorch Gradient 默认累加的后果；
7. 使用 `torch.no_grad()` 做最小评估；
8. 区分 `model.eval()` 与 `torch.no_grad()`；
9. 看懂论文/开源代码中最常见的训练循环；
10. 为后面的 Dataset/DataLoader、SGD、AdamW、Learning Rate 打好接口基础。

1 【原理】训练到底在重复什么？

一个最小监督学习训练 Step，可以写成：

清旧梯度 → Forward → Loss → Backward → 更新参数

对应 PyTorch：

```
optimizer.zero_grad()
pred = model(x)
loss = criterion(pred, y)
loss.backward()
optimizer.step()
```

一个标准 PyTorch Training Step 的职责链

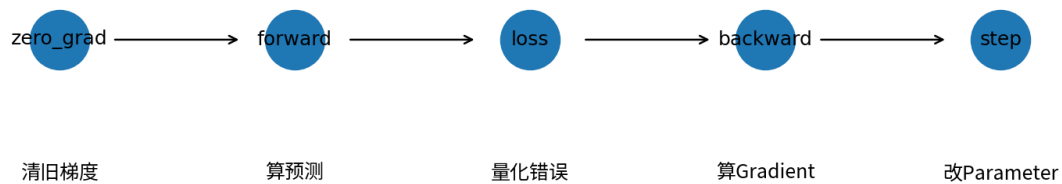


图 1: 一个标准 PyTorch Training Step 的职责链

训练不是一个叫“train”的神秘操作，而是同样的五步被反复执行。模型能学会东西，是因为每轮最后 Parameter 都发生了一点变化，而下一轮 Forward 用的是更新后的 Parameter。

2 【逐步拆解】五行代码分别做什么？

1. optimizer.zero_grad(): 清理上一轮 Gradient

第 29 讲已经验证，PyTorch 的 .grad 默认是累加：

$$\text{grad}_{\text{new}} = \text{grad}_{\text{old}} + \text{本次 backward 的贡献}$$

所以一般训练每个 Step 开头要先清理旧梯度。

本讲实验中，同一个 Loss 连续 backward 两次但不清零：

$$\frac{\partial L}{\partial w} : -7.1132 \rightarrow -14.2265$$

刚好变成 2 倍。

梯度累加本身不是“错误机制”。以后做 Gradient Accumulation 时会故意利用它。错误在于：你本来想每个 mini-batch 独立更新，却忘了清理上一轮梯度。

常见写法：

```
optimizer.zero_grad()
# 或
optimizer.zero_grad(set_to_none=True)
```

`set_to_none=True` 会把 `grad` 设为 `None`；默认行为和版本/接口设置可能不同。对零基础阶段，先掌握“每轮 `backward` 前确保旧梯度不干扰本轮”即可。

2. `pred = model(x)`: Forward

Forward 使用**当前参数**计算预测：

$$\hat{y} = f(x; \theta)$$

其中 θ 表示所有可训练参数。

这一步通常同时让 Autograd 记录计算依赖：

$$Parameter \rightarrow Intermediate\ Tensor \rightarrow Prediction \rightarrow Loss$$

但此时还没有真正求出 Gradient。

3. `loss = criterion(pred,y)`: 把错误压成优化目标

例如回归任务：

$$L = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$$

分类/LLM 则常见 Cross Entropy。

Optimizer 并不知道“答案是否正确”。它只看到 Parameter 的 Gradient。Loss Function 是连接“任务目标”和“梯度优化”的桥梁。

4. `loss.backward()`: 计算 Gradient，不修改 Parameter

Backward 从 Loss 沿计算图反向执行 Chain Rule，把结果写入：

```
parameter.grad
```

本讲实际验证：执行 `backward()` 后，Weight/Bias 数值没有变化；变化的是它们的 `.grad`。

5. `optimizer.step()`: 真正修改 Parameter

以最简单的 SGD 为例，可近似理解为：

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

其中：

- θ : Parameter;
- $\nabla_{\theta} L$: 由 `backward` 得到的 Gradient;
- η : Learning Rate。

`optimizer.step()` 不会自己重新计算 Gradient。它使用 Parameter 当前已经存储的 `.grad`。所以如果没有正确执行 `backward`, `step` 就没有正确的更新信息。

3 【具体数字实验】一次 Step 中 Parameter 什么时候真正变化?

本讲使用最小模型:

$$\hat{y} = wx + b$$

数据满足:

$$y = 2x + 1$$

固定初始化得到:

$$w \approx -0.54044, \quad b \approx -0.15630$$

第一次 Forward:

$$L \approx 10.37241$$

Backward 得到:

$$\frac{\partial L}{\partial w} \approx -7.11323, \quad \frac{\partial L}{\partial b} \approx -2.31260$$

此时 Parameter 本身仍未改变。

执行:

```
optimizer.step()
```

之后:

$$w \approx -0.18478, \quad b \approx -0.04067$$

再 Forward 一次:

$$L \approx 7.76556$$

因此应明确区分: Backward 产生“修改建议” (Gradient); Step 才根据这些建议真正改 Parameter。

4 【完整训练】模型怎样从随机参数学到 $w = 2, b = 1$?

标准循环:

```
for step in range(120):
    optimizer.zero_grad()
    pred = model(X)
    loss = criterion(pred, Y)
    loss.backward()
    optimizer.step()
```

实际训练轨迹:

Step	Loss	Weight	Bias
0	10.372410	-0.184780	-0.040671
9	0.798981	1.437796	0.596823
29	0.004401	1.972466	0.950983
59	0.0000055	1.999702	0.997922
119	1.70×10^{-11}	2.000000	0.999996

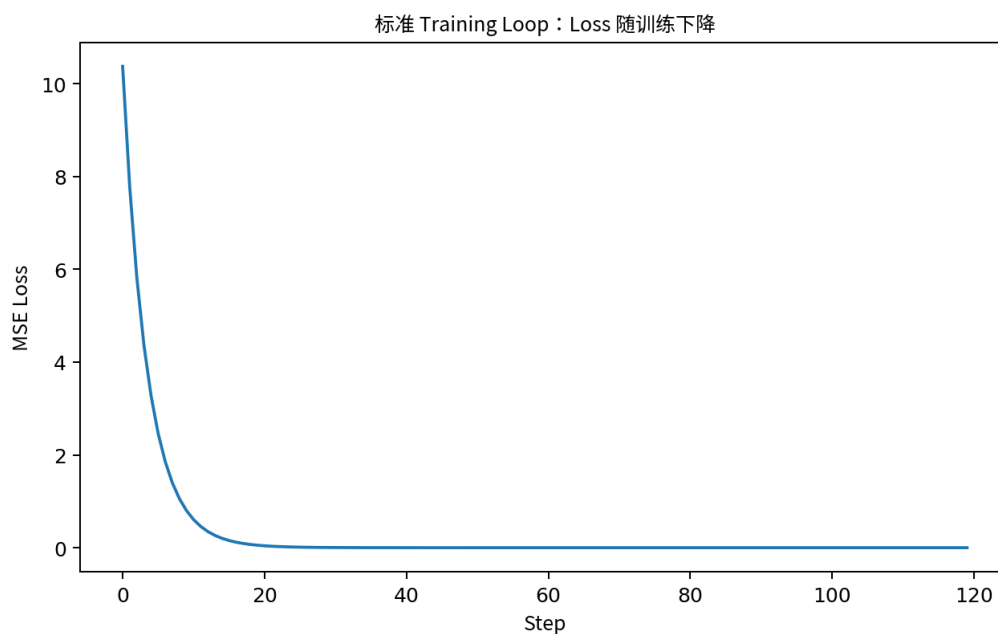


图 2: 标准 Training Loop 中 Loss 随 Step 下降

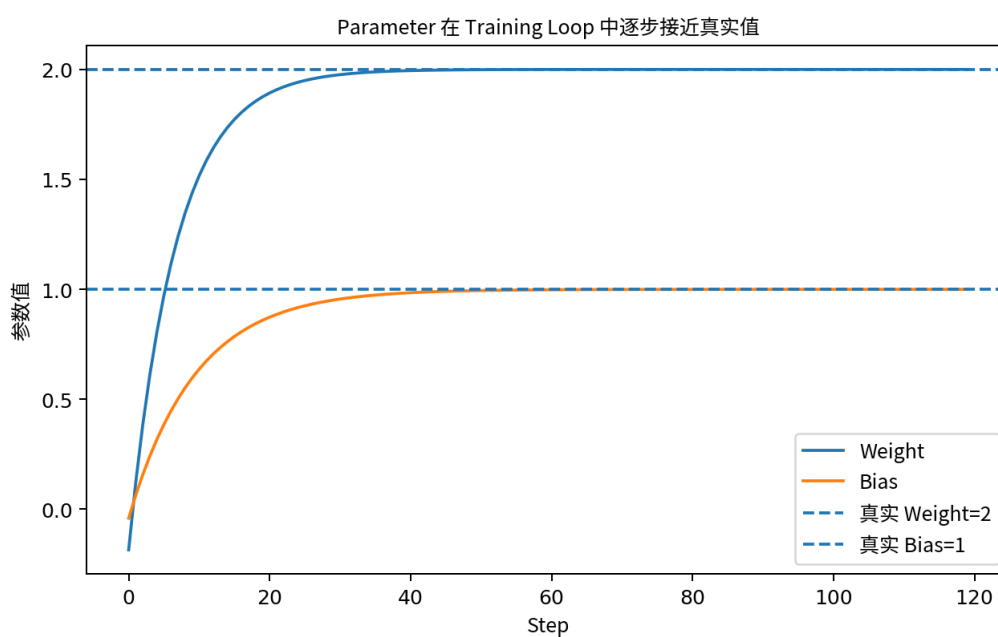


图 3: Weight 与 Bias 逐步接近真实参数 2 和 1

所谓“模型学习”，在这个最小例子里完全可以落到数字层面理解：每一轮都重新计算预测和 Loss，用 Backward 得到当前局部梯度，再由 Optimizer 修改 Parameter。

5 【反例实验 1】不 zero_grad() 会怎样？

相同模型、相同数据、参数也没有修改：

$$g_w^{(1)} = -7.11323$$

第二次重新 Forward/Backward，但不清梯度：

$$g_w^{stored} = -14.22647$$

也就是：

$$\frac{-14.22647}{-7.11323} = 2$$

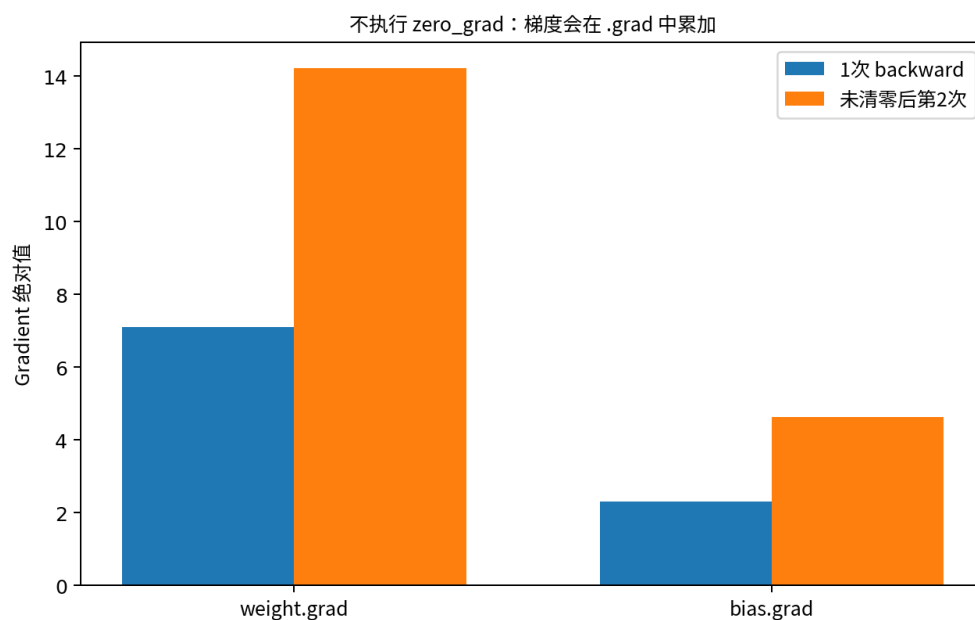


图 4: 不清梯度时，新的 Gradient 会累加进已有.grad

6 【反例实验 2】没有 backward(), 只 step() 呢?

实验先明确：

```
weight.grad = None
bias.grad = None
```

然后直接：

```
optimizer.step()
```

结果 Weight 与 Bias 都没有变化。

原因很简单：Optimizer 不是“自己知道答案”的东西；它依赖 Gradient。

7 【反例实验 3】只有 backward(), 没有 step() 呢?

连续五次执行：

```
zero_grad()
forward
```

```
loss
backward()
# 故意没有 step()
```

每轮 Loss 都是：

10.37240982

Parameter 完全没有变化。

这说明 `loss.backward()` 只是“测量当前参数应该往哪里改”；没有 Parameter Update，就不发生真正训练。

8 【Evaluation】`torch.no_grad()` 是干什么的？

模型训练完成后，评估/推理通常不要求梯度：

```
with torch.no_grad():
    pred = model(x)
```

本讲实际验证，同一个输入：

- 普通 Forward 输出 = 1.9999961853；
- `no_grad()` 输出 = 1.9999961853；
- 数值差 = 0；
- 但前者 `requires_grad=True`，后者为 `False`。

`model.eval()` 与 `torch.no_grad()` 不是一回事

机制	主要作用	是否关闭 Autograd 记录
<code>model.eval()</code>	把 Module 切换到 evaluation mode；以后 Dropout/BatchNorm 等会受影响	否
<code>torch.no_grad()</code>	暂时关闭 Gradient tracking	是

以后做验证/推理，经常同时写 `model.eval()` 与 `torch.no_grad()`。前者控制某些层的行为，后者控制是否构建梯度图。两者职责不同。

9 【代码阅读】真实训练代码应该按什么顺序看？

看到陌生训练脚本，可以先找这五个锚点：

1. Model: `model = ...`;
2. Loss: `criterion / loss_fn`;

3. **Optimizer**: `torch.optim...`;
4. **Training Loop**: batch 从哪里进来;
5. **五步闭环**: `zero grad / forward / loss / backward / step`。

然后再看复杂部分: `scheduler`、`mixed precision`、`gradient clipping`、`accumulation`、`distributed training` 等。

【科研检查点】如果论文报告“训练失败/不稳定”，不要只看最终指标。最低应检查：`Loss` 是否下降、`Gradient` 是否为 `None/NaN/Inf`、`Learning Rate`、是否正确 `zero grad`、哪些 `Parameter` 实际发生更新、`train/eval mode` 是否正确。很多所谓“模型问题”首先可能是训练闭环问题。

10 【与 LLM / Transformer / Agent Systems 的关系】

LLM 预训练虽然巨大，但最小逻辑仍然是：

Token IDs → Transformer → Logits → CrossEntropy → Backward → Optimizer.step

区别主要在于：

- `Parameter` 数量从几个变成数十亿甚至更多；
- Batch 是大量 Token；
- `Optimizer` 常用 `AdamW`；
- 可能增加 `Gradient Accumulation`、`Mixed Precision`、`Distributed Training`；
- 还可能使用 `Learning Rate Scheduler`、`Gradient Clipping` 等。

但底层闭环没有变。

对于 `Agent Systems`，大多数情况下你不会从零训练基础模型，但你会遇到：微调、`LoRA`、`reward model`、`reranker`、`embedding model`、`policy/value model` 或其他可训练组件。判断实验是否可信时，仍要能读懂这个训练闭环。

11 【常见误区】

1. **误区**：`backward()` 会自动更新参数。错。它主要计算并累积 `Gradient`。
2. **误区**：`step()` 会自己求梯度。错。它读取当前 `Parameter` 的 `.grad`。
3. **误区**：`Gradient` 会自动覆盖。错。PyTorch 默认累加。
4. **误区**：`no_grad()` 就等于 `eval()`。错，二者职责不同。
5. **误区**：训练 `Loss` 下降就证明模型泛化好。错，只能先证明训练优化在当前数据上发生了。
6. **误区**：一定只能在 Step 开头 `zero grad`。更准确地说，只要在下一次需要独立累积 `Gradient` 之前清理即可；放在 Step 开头最容易读懂。

本讲最低闭环

看到下面代码：

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

你应该能逐行说出：

清上一轮梯度；用当前参数算预测；把预测误差压成 Loss；沿计算图计算各 Parameter Gradient；Optimizer 根据 Gradient 与自己的更新规则修改 Parameter。

下一讲预告

第 32 讲将进入 Dataset、DataLoader 与 Mini-Batch：为什么真实训练不把全部数据一次塞进模型，Batch Size、shuffle、一个 Step 到底对应多少样本，以及 PyTorch 怎样把数据自动分批送入上面的 Training Loop。